Tema 4: Interfaces Gráficas con Swing
JList en Java

Pablo Gordillo (pabgordi@ucm.es)

Departamento de Sistemas Informáticos y Computación

Facultad de Informática



(Basado en el material creado por Puri Arenas, Jesús Correas y Samir Genaim)

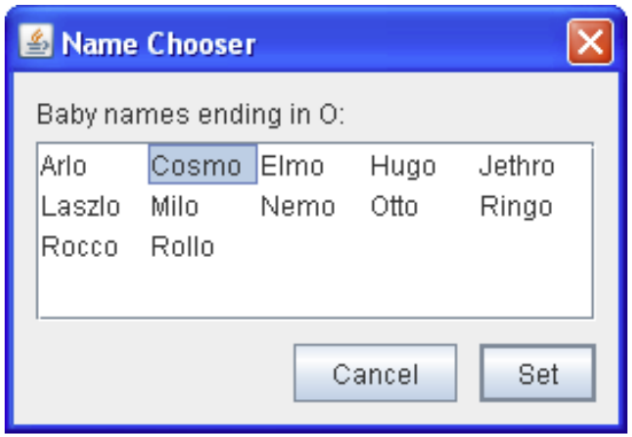
JList

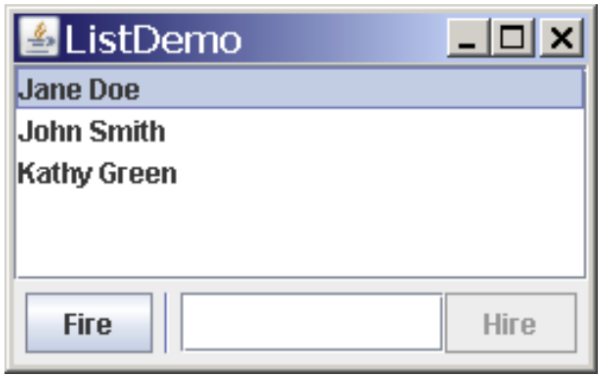
- Agrupa un conjunto de elementos en una o más columnas.
- Un JList está implementado como un genérico. Para definir su tipo:

JList<Tipo> list;

- Para crearlo hay que ejecutar

list = new JList(Tipo[])





JList

```
JList <String> boxes;
String names[] = {"Red","Blue","Green","Orange",
  "Yellow","Pink","Cyan","Gray",
  "Black","Magenta"};
boxes = new JList<String>(names);
```




```
boxes.setFixedCellHeight (20);
boxes.setFixedCellWidth(140);
```



JList

boxes.setSelectionMode(ListSelectionModel.SINGLE_SELECTION);



JList

boxes.setSelectionMode(ListSelectionModel.SINGLE_INTERVAL_SELECTION);
(Pulsando CTRL y el ratón).



JList

boxes.setSelectionMode(ListSelectionModel.MULTIPLE_INTERVAL_SELECTION);
(Pulsando CTRL y el ratón).



JList

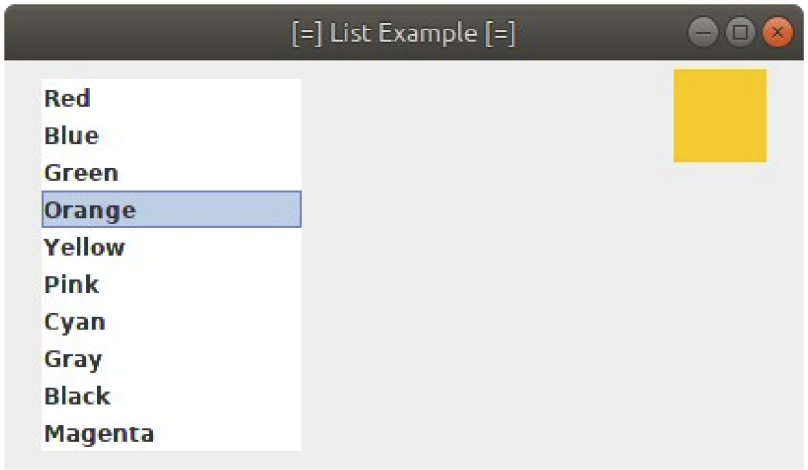
- Cuando se selecciona un elemento de una lista, entonces se lanza un evento del tipo ListSelection.
- Para tratar dicho evento hay que añadir un "list selection listener" a la componente (**addListSelectionListener**). Un "list selection listener" debe implementar el método:

  valueChanged(ListSelectionEvent e).
- El método **getValuesAdjusting()** devuelve true si el usuario está todavía seleccionando una acción con el ratón, pero aún no se ha decidido.
- El método **getSelectedIndex()** devuelve el índice seleccionado por el usuario en la lista.

Ejemplo

```
public class ListExample1 extends JFrame
                          implements ListSelectionListener {

    JList <String> boxes; // se define JList
```



Ejemplo

```
public class ListExample1 extends JFrame
                          implements ListSelectionListener {

    JList <String> boxes; // se define JList
    JPanel[] boxArray = new JPanel[10];
    // nombres que van a aparecen en JList
    String names[] = { "Red", "Blue", "Green", "Orange",
      "Yellow", "Pink","Cyan", "Gray",
      "Black", "Magenta" };

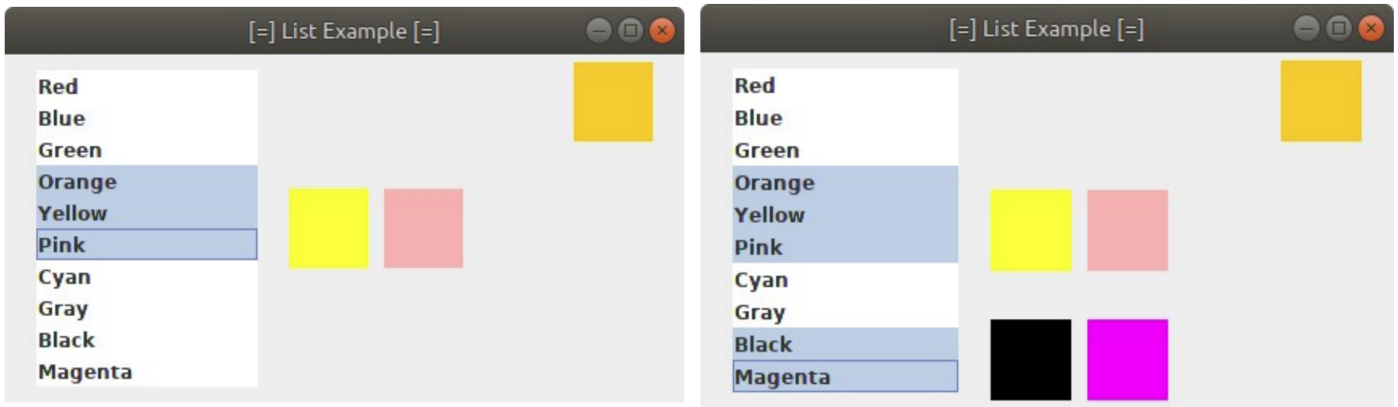
    private void initGUI() {
      ...
      boxes = new Jlist <String>(names); // se crea JList
      boxes.setFixedCellHeight (20); // se fija el alto de celda
      boxes.setFixedCellWidth(140); // se fija el ancho de celda
      // selección simple (solo se puede seleccionar uno)
      boxes.setSelectionMode(ListSelectionModel.SINGLE_SELECTION);
      // se añade un listener
      boxes.addListSelectionListener (this );
      ...
    }
```

Ejemplo

```
// se ejecuta cuando usamos el ratón y el teclado
public void valueChanged(ListSelectionEvent e) {
  // se comprueba que el usuario ha dejado de seleccionar
  if (e.getValueIsAdjusting() == false) {
    // se comprueba que la componente es el JList
    if (e.getSource() == boxes) {
      for (int i = 0; i < 10; i++) {
        // se recorre el array de paneles para encontrar
        // que panel coincide con el índice seleccionado
        if (i == boxes.getSelectedIndex())
        // se pone el panel visible si el panel número i
        boxArray[i].setVisible(true);
        // se oculta el panel si es distinto del número i
        else boxArray[i].setVisible(false);
      }
    }
  }
}
```

Ejemplo

Si usamos una de las siguientes opciones:
boxes.setSelectionMode(ListSelectionModel.SINGLE_INTERVAL_SELECTION);
boxes.setSelectionMode(ListSelectionModel.MULTIPLE_INTERVAL_SELECTION);



Ejemplo

Si usamos una de las siguientes opciones:
boxes.setSelectionMode(ListSelectionModel.SINGLE_INTERVAL_SELECTION);
boxes.setSelectionMode(ListSelectionModel.MULTIPLE_INTERVAL_SELECTION);

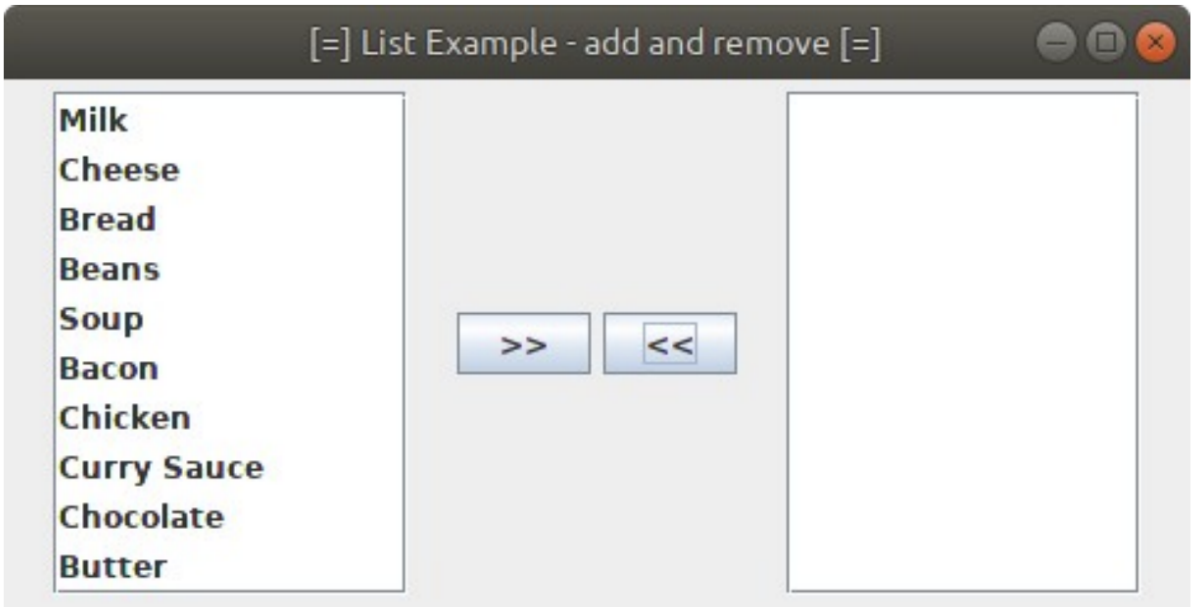
```
public void valueChanged(ListSelectionEvent e) {
  if (e.getValueIsAdjusting() == false) {
    if (e.getSource() == boxes) {
      // se coge el array de índices seleccionados
      int [] fromindex = boxes.getSelectedIndices();
      // se ocultan todos los paneles
      for (int i = 0; i < 10; i++){
        boxArray[i].setVisible (false);
      }
      // se hacen visibles los paneles seleccionados que
      // están en "fromindex"
      for (int i = 0; i < fromindex.length; i++){
        boxArray[fromindex[i]].setVisible (true);
      }
    }
  }
}
```

JList: Añadir y Borrar elementos

- Al igual que para las tablas, para manipular los datos de una lista (añadir, eliminar, ..., elementos) necesitamos usar un modelo que en este caso extiende a la clase **DefaultListModel**.
- Después la lista debe crearse pasando como parámetro el modelo:

```
DefautListModel listModel = DefaultListModel<String>();
Jlist list = new Jlist(listModel);
listModel.addElement(String);
listModel.remove(String);
```

JList: Añadir y Borrar elementos



Tenemos dos listas. Pasamos elementos de una lista a otra a través de los botones. Entonces añadimos y eliminamos elementos de la lista.

JList: Añadir y Borrar elementos

```
public class ListExample2 extends JFrame implements
        ActionListener {

  // un modelo para cada lista
  JList <String> itemList, shoppingList ;
  // un botón para comprar y otro para devolver
  JButton buttonin, buttonout;
  // un modelo para cada lista
  DefaultListModel<String> shopping, items;
  // productos del modelo "shopping"
  String shoppingItems[] = { "Milk", "Cheese", "Bread",
    "Butter", "Beans", "Soup", "Bacon",
    "Chicken", "Curry Sauce", "Chocolate" };
```

JList: Añadir y Borrar elementos

```
private void initGUI() {
  ...
  // se crean los modelos
  shopping = new DefaultListModel<String>();
  items = new DefaultListModel<String>();
  // se inicializa el modelo de compra con los productos
  for (int i = 0; i < shoppingItems.length; i++) {
    shopping.addElement(shoppingItems[i]);
  }
  // se cran las listas con el modelo correspondiente
  itemList = new JList<String>(items);
  shoppingList = new JList<String>(shopping);
  ...
}
```

JList: Añadir y Borrar elementos

```
public void actionPerformed(ActionEvent e) {
  int i = 0;
  // pulsamos botón comprar
  if (e.getSource() == buttonin) {
    // cogemos los índices de los productos seleccionados
    int [] fromindex = itemList.getSelectedIndices ();
    // cogemos los productos seleccionamos
    List <String> from = itemList.getSelectedValuesList ();
    // añadimos los productos seleccionados a la lista de compra
    for (String s : from) items.addElement(s);
    // eliminamos los productos seleccionados de los productos
    // posibles a comprar
    for (i = (fromindex.length - 1); i >= 0; i--)
    shopping.remove(fromindex[i]);
  }
  ...
}
```

JList con Modelo

- Al igual que para las tablas, podemos definir nuestro propio modelo.
- Para ello los elementos (contenido) se almacenarán en una nueva clase que aportará las operaciones para manipularlos. Así nos creamos nuestro propio modelo, en lugar de utilizar **DefaultListModel** o similares, usará nuestro propio modelo.
- La nueva clase que extiende nuestro modelo debe extender la clase **AbstractListModel<T>**, que ofrece funcionalidad para registrar y notificar a los observadores, al igual que para las tablas.

JList con Modelo

```
class MyListModel extends AbstractListModel<String> {
  ArrayList <String> x = new ArrayList<String>();

  // determina el tamaño de la lista
  @Override
  public int getSize() { return x.size(); }

  // determina el tipo y lugar de los elementos de la lista
  @Override
  public String getElementAt(int index) { return x.get(index);}

  // aãde elementos a nuestro modelo
  public void addElement(String s) {
    x.add(s);
    fireContentsChanged(this, x.size()-1, x.size()-1);
  }

  // elimina elementos del modelo
  public void remove(int i) {
    x.remove(i);
    fireContentsChanged(this, i, i);
  }
}
```

JList con Modelo

```
private void initGUI() {
  ...
  shopping = new MyListModel();
  items = new MyListModel();
  for (int i = 0; i < shoppingItems.length; i++) {
    shopping.addElement(shoppingItems[i]);
  }
  itemList = new Jlist<String>(items);
  shoppingList = new Jlist<String>(shopping);
  ...
}
```

JList con Modelo

```
public void actionPerformed(ActionEvent e) {
  int i = 0;
  if (e.getSource() == buttonout) {
    List <String> to = itemList.getSelectedValuesList ();
    int [] toindex = itemList.getSelectedIndices ();
    for (String s : to) shopping.addElement(s);
    for (i = (toindex.length - 1); i >= 0; i--)
    items.remove(toindex[i]);
  }
  ...
```